Programming and Systems Development

Mock Lab Exam B - Python Part (40 marks)

Time allowed for this mock: 1 hour (the same as the real mid-term test). Open book: you may use your printed notes and practice materials. No communication with other candidates. No Tkinter/GUI is needed. This paper uses pandas, numpy, matplotlib and sqlite3 only (no PyTorch).

You must write the code for ALL tasks in the SAME Python source file.

Scenario: Module Score Logger. A tutor records student test scores in a CSV file `module_scores.csv` with the columns `Student, Module, Score, Hours`:

- `Score` is a whole number from 0 to 100.
- `Hours` is the study time (a positive decimal number, e.g. 5.5).

A starter file `module_scores.csv` is provided. Place it in the same folder as your program.

Python Task 1: Console Data Logger [14 marks]

Task 1a: File setup [4 marks]

Your program must use the file `module_scores.csv`.

- If the file does **not** exist, create it with the header row `Student,Module,Score,Hours` and print a message saying it was created.
- If the file **already exists**, do not overwrite it; print a message saying it is open for appending. [4 marks]

Task 1b: Data entry [5 marks]

Prompt the user to enter records one at a time (student name, module, score, study hours). The user stops by typing `q` (or `Q`) as the student name. Each valid record must be **appended** as a new row at the end of the file. Student and module names are stored with a capital first letter (so `maths` is stored as `Maths`). After the loop print `Data saved to module_scores.csv`. [5 marks]

Task 1c: Input validation [5 marks]

Use `try`/`except` so the program never crashes on bad input:

- The score must be a whole number from 0 to 100; hours must be a positive number. If the input is invalid, print an error message and ask for that value again.
- An empty student name must be rejected with a message.

When testing your code, type EXACTLY the interaction below (a copy is supplied in the file `stdin.txt`). The later tasks depend on these records.

```
=== Task 1: CSV File Setup ===
module_scores.csv already exists and is open for appending

=== Task 1: Data Entry ===
Enter student name (or 'q' to quit): Mei
Enter module: physics
Enter score (0–100): 74
Enter study hours: 5.5
Record saved: Mei, Physics, 74, 5.5
Enter student name (or 'q' to quit): Dara
Enter module: maths
Enter score (0–100): abc
Invalid score – please enter a whole number from 0 to 100.
Enter score (0–100): 105
Invalid score – please enter a whole number from 0 to 100.
Enter score (0–100): 82
Enter study hours: –2
Invalid hours – please enter a positive number.
Enter study hours: x
Invalid hours – please enter a positive number.
Enter study hours: 7.5
Record saved: Dara, Maths, 82, 7.5
Enter student name (or 'q' to quit):
This cannot be empty.
Enter student name (or 'q' to quit): Kofi
Enter module: biology
Enter score (0–100): 72
Enter study hours: 6
Record saved: Kofi, Biology, 72, 6.0
Enter student name (or 'q' to quit): Zoe
Enter module: chemistry
Enter score (0–100): 97
Enter study hours: 9
Record saved: Zoe, Chemistry, 97, 9.0
Enter student name (or 'q' to quit): Q

Data saved to module_scores.csv
```

(The empty name prompt after `Dara`'s record is you pressing Enter with nothing typed. Your wording of the messages may differ slightly.)

Python Task 2: Summary Statistics and Visualisation [12 marks]

After the data entry has finished, use pandas, numpy and matplotlib.

Task 2a: Read and clean [3 marks]

Read the CSV file into a DataFrame, remove exact duplicate rows, and print the number of records that remain and the number of different modules. [3 marks]

Task 2b: Statistics with numpy [3 marks]

Using numpy functions on the `Score` column, print the mean, median, (population) standard deviation, minimum and maximum, each to 2 decimal places where it is a decimal. [3 marks]

Task 2c: Save the summary [2 marks]

Save these results (and the record count) in a text file called `score_summary.txt`. Re-running the program must not make the file grow: it must hold the same lines each time when no new data is added. [2 marks]

Task 2d: Figure with two subplots [4 marks]

Create ONE matplotlib figure with two subplots side by side:

- Left: a **histogram** of the scores. Title "Score Distribution", x label "Score", y label "Frequency".
- Right: a **bar chart** of the mean score for each module. Title "Mean Score per Module", sensible axis labels.

Display the figure with `plt.show()`. [4 marks]

Python Task 3: SQLite Database [14 marks]

Task 3a: Create the database and table [4 marks]

Create an SQLite database named `ScoresDB` and a table named `Results` with columns for `Student`, `Module`, `Score` and `Hours`, using sensible data types. [4 marks]

Task 3b: Insert the data [4 marks]

Insert the records from the CSV file (after the cleaning in Task 2a) into `Results`. No duplicate entries may be added, even if the whole program is run several times. [4 marks]

Task 3c: Queries [6 marks, 2 each]

Write and run SQL queries, printing neat results, for:

1. the total number of records in `Results`;
2. the average score for each module, highest average first;
3. the top 3 scores: student, module and score, highest first.

Example layout of the output (placeholders only):

```
Records in Results: 00
Average score per module:
  ModuleName 00.00
Top 3 scores:
  StudentName ModuleName 00
```

Submission

Ensure your submission file is named exactly as instructed, i.e. `your_name_python_task.py`, and upload that single file. Marks are also awarded for clear naming, helpful comments and tidy layout.

Task	Marks
1a File setup	4
1b Data entry loop	5
1c Validation	5
2a-2d Statistics and figure	12
3a-3c SQLite	14
Total	**40**